PROGRAM

single linked list

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
	int data;
	struct node *next;
};
struct node *head,*newnode,*curr,*prev,*temp;
void prepend(int data) {
	newnode = (struct node*)malloc(sizeof(struct node));
	newnode->data = data;
	newnode->next = NULL;
	if(head == NULL)
	{
		head = newnode;
	}
	else
	{
		newnode->next = head;
		head = newnode;
	}
	printf("data prepended:%d \n",data);
}
void append(int data) {
	newnode = (struct node*)malloc(sizeof(struct node));
	newnode->data = data;
	newnode->next = NULL;

	if(head == NULL)
	{
		head = newnode;
	}
	else
	{
		struct node *temp = head;
		while(temp->next != NULL) //navigating to last node
		{
			temp = temp->next;
		}
		temp->next = newnode;
	}
	printf("data appended:%d \n",data);
}
void display()
```

```
{
   temp = head;
        while(temp != NULL)
        {
                printf(" %d -> ",temp->data);
                temp = temp->next;
        }
}
void insert(int data,int pos)
{

        newnode = (struct node*)malloc(sizeof(struct node));
        newnode->data = data;
        newnode->next = NULL;
        if(pos == 0)
        {
                newnode->next = head;
                head = newnode;
        }
        else
        {
                int i=0;
                curr = head;
                prev = NULL;

                while(curr != NULL)
                {
                        if(i == pos)
                        {
                                break;
                        }
                        prev = curr;
                        curr = curr->next;
                        i++;
                }

                newnode->next = curr;
                prev->next = newnode;

        }
        printf("data inserted:%d at position:%d \n",data,pos);
}

void search(int data)
{
        temp = head;
```

```
	int found=0,pos=1;
	while(temp != NULL)
	{
		if(temp->data == data)
		{
			found=1;
			break;
		}
		temp = temp->next;
		pos++;
	}
	if(found == 0)
	{
		printf("data not found");
	}
	else
	{
		printf("data found @:%d",pos);
	}
}
void delete(int data)
{
	int found =0;
	prev = NULL;
	temp = head;

	while(temp != NULL)
	{
		if(temp->data == data)
		{
			found = 1;
			break;
		}
		prev = temp;
		temp = temp->next;
	}

	if(found == 0)
	{
		printf("\n element not found");
	}
	else
	{
		if(temp == head) //delte head node
		{
			head = temp->next;
```

```
                }
                else if(temp->next == NULL) //delete last node
                {
                        prev->next = NULL;
                }
                else //mid node
                {
                        prev->next = temp->next;
                }
                printf("\nelement delted:%d\n",temp->data);
                free(temp);
        }

}
int main()
{
        append(10); //pos=0
        append(20);//pos=1
        append(30);//pos=2
        display();

        printf("\n");
        prepend(40);
        display();

        delete(20);
        printf("\n");
        display();

        printf("\n");
        insert(50,0);
        display();
        printf("\n search element: 40 \n");
        search(40);
        printf("list data\n ");
        display();

}
```

Output:

```
data appended:10
data appended:20
data appended:30
 10 ->  20 ->  30 ->

data prepended:40
 40 ->  10 ->  20 ->  30 ->

element delted:20
 40 ->  10 ->  30 ->


data inserted:50 at position:0
 50 ->  40 ->  10 ->  30 ->

search element: 40
data found @:2

list data:
50 ->  40 ->  10 ->  30 ->
```